

07: CLI/SDK, REST, Concurrency (Python/Go/Rust)

Cloud GPU teams ship their APIs to customers through SDKs and command-line tools, so interviews for these roles ask you to design a REST API, explain how a good SDK behaves, and write working client code in the language you list as your strongest. This chapter covers the API design, the structure of an SDK in Python, Go, and Rust, the concurrency patterns that client code depends on, and the role of code generation.

This chapter covers

- Designing a resource-oriented REST API with asynchronous operations
- The behavior every cloud SDK needs: authentication, retries, pagination, and cancellation
- Idiomatic SDK structure in Python, Go, and Rust
- Rate limiting, worker pools, cancellation, and backpressure in client code
- Where code generation fits in an SDK and operator toolchain

Contents

1. REST API design for a GPU cloud service.....	2
Long-running operations	2
What an SDK must do	2
2. A Python SDK.....	4
3. A Go SDK.....	5
4. A Rust SDK and CLI	6
5. Concurrency patterns	7
Token bucket rate limiter	7
Worker pool	7
Timeouts and cancellation.....	7
Backpressure and retries.....	7
Patterns across services	8
6. Code generation	9
Exercises	10
Summary	11

1. REST API design for a GPU cloud service

Model the API around resources. Each resource has a collection path and an item path, and HTTP methods express the operation:

Method	Path	Purpose
GET	/v1/gpu-workloads	List workloads, one page at a time
POST	/v1/gpu-workloads	Create a workload (asynchronous)
GET	/v1/gpu-workloads/{id}	Get a workload and its status
PATCH	/v1/gpu-workloads/{id}	Update a workload, for example to change the replica count
DELETE	/v1/gpu-workloads/{id}	Delete a workload (asynchronous)
GET	/v1/gpu-workloads/{id}/metrics	GPU and network metrics for a workload
GET	/v1/gpu-types	The catalog of available GPU types

The version in the path (/v1) lets you introduce incompatible changes under /v2 while existing clients continue to work.

Long-running operations

Creating a GPU workload can take minutes, so the server accepts the request and returns immediately. The client sends an idempotency key with the request:

```
POST /v1/gpu-workloads HTTP/1.1
Content-Type: application/json
Idempotency-Key: 7c9e6679-7425-40de-944b-e07fc1f90ae7

{"name": "demo", "gpuType": "A100", "gpuCount": 4, "region": "us-central"}
```

The server responds with 202 Accepted and the location of an operation resource that the client can poll:

```
HTTP/1.1 202 Accepted
Location: /v1/operations/op-123
```

If the client does not receive the response and retries with the same Idempotency-Key, the server returns the original operation instead of creating a second workload.

What an SDK must do

An SDK is more than a set of HTTP calls. A well-behaved SDK does the following:

Behavior	Why
Authenticates once and refreshes tokens before they expire	Callers should not handle credentials on every request
Retries on 429 and 5xx responses with exponential backoff and jitter	Transient failures are common in distributed systems
Retries non-idempotent requests only with an idempotency key	A retried POST must not create a duplicate resource
Honors the Retry-After header	The server knows when it will accept requests again
Waits for long-running operations when asked	Callers can choose between returning immediately and waiting for completion
Returns typed errors	Callers can distinguish "not found" from "quota exceeded" without parsing messages
Exposes pagination as an iterator	Callers can loop over all items without handling page tokens
Supports timeouts and cancellation on every call	A caller must be able to stop waiting

2. A Python SDK

A typical package layout separates transport, models, and resources:

```
gpucloud/  
├─ client.py          # HTTP session, auth, retries  
├─ models.py          # dataclasses/Pydantic models  
├─ operations.py      # async operation polling  
├─ gpu_workloads.py   # resource methods  
└─ exceptions.py
```

Implementation choices:

- **HTTP client.** Use one shared `requests.Session`, or `httpx.Client` and `httpx.AsyncClient` if you support both synchronous and asynchronous use. Sharing the client reuses connections.
- **Retries.** Use `tenacity`, or a small retry function, for 429 and 5xx responses.
- **Models.** Use `dataclasses` or `Pydantic` models so that responses have types and are validated.
- **Resource access.** Group methods by resource: `client.gpu_workloads.create(...)`.
- **Pagination.** Return a generator that requests the next page when the caller reaches the end of the current one.
- **Configuration.** Find configuration and credential files with `platformdirs`, so paths are correct on Linux, macOS, and Windows.

3. A Go SDK

A Go SDK typically exposes a top-level client with a service field for each resource group, configured with functional options:

```
type Client struct {
    baseURL    string
    http       *http.Client
    token      TokenProvider
    Workloads *WorkloadsClient
}

func New(options ...Option) (*Client, error)

func (w *WorkloadsClient) Get(ctx context.Context, id string) (*Workload, error)
```

Conventions to follow:

- **Pass** `context.Context` **as the first argument of every call** that performs I/O, so callers control cancellation and deadlines.
- **Configure** `http.Client` **explicitly**. The default client has no timeout. Set timeouts on the client and transport, and reuse one client for connection pooling.
- **Use functional options** (`WithToken`, `WithRetries`, `WithUserAgent`) so that new settings do not change the constructor's signature.
- **Return errors that work with** `errors.Is` **and** `errors.As`, such as a sentinel `ErrNotFound` or an `*APIError` type with the status code.
- **Generate the client from OpenAPI** with a tool such as `oapi-codegen`, or write it by hand when you need more control over the interface.

4. A Rust SDK and CLI

Rust is a good fit for command-line tools: it produces fast, self-contained binaries that are distributed through cargo or as release downloads. The common crates are:

Need	Crate
HTTP	request (async)
JSON	serde, serde_json
Command-line parsing	clap with the derive API
Async runtime	tokio
Errors	thiserror for library error types, anyhow for applications
Table output	comfy-table
Configuration directories	directories
Testing	#[test], and assert_cmd for testing the CLI binary

Design the command structure around resources and verbs, and support a machine-readable output format:

```
gpucloud create workload --name demo --gpu A100 --count 4 --region us-central
gpucloud get workload demo -o json
gpucloud logs demo --follow
```

12-rust-guide-part2-errors-concurrency-sdk.md covers Rust error handling and SDK structure in more detail.

5. Concurrency patterns

Client code for cloud services depends on a small set of concurrency patterns. Each has an implementation in `rust-interview-lab`.

Token bucket rate limiter

A token bucket holds up to capacity tokens and refills at rate tokens per second. Each request takes one token; a request that finds the bucket empty waits or is rejected. The capacity sets the largest allowed burst, and the rate sets the sustained throughput. In Rust, protect the bucket's state with a mutex, or use atomics for a lock-free version. See `src/problems/rate_limiter.rs`.

Worker pool

A worker pool has a queue of jobs, a fixed number of workers that take jobs from the queue, and a channel for results. The fixed number of workers limits concurrency, for example to avoid opening hundreds of connections to an API at once. A CLI that operates on many resources, such as deleting 500 workloads, uses this pattern. See `src/problems/worker_pool.rs`.

Timeouts and cancellation

Every language provides a way to stop waiting:

Language	Timeout	Cancellation
Rust	<code>tokio::time::timeout</code>	<code>tokio_util::sync::CancellationToken</code> , or dropping the future
Go	<code>context.WithTimeout</code>	<code>context.WithCancel</code>
Python	<code>asyncio.wait_for</code> , <code>asyncio.timeout</code>	<code>Task.cancel()</code>

Backpressure and retries

- **Use bounded channels and queues.** When a bounded queue is full, producers wait, and the system slows down instead of consuming unbounded memory. An unbounded channel removes that protection.
- **Add jitter to retry delays.** If many clients retry at the same fixed intervals, their retries arrive together and overload the server again.
- **Honor** `Retry-After`.
- **Use idempotency keys** for POST and other methods that are not idempotent.

Patterns across services

Pattern	Use
Distributed rate limiting (a token bucket or GCRA in Redis)	Enforcing one limit across many API servers
Leaky bucket	Smoothing request bursts into a constant rate
Consistent hashing	Assigning keys to shards so that adding a shard moves few keys; see <code>src/problems/consistent_hash.rs</code>
Transactional outbox	Publishing an event reliably when a database record changes

6. Code generation

Code generation keeps clients, servers, and documentation consistent with one specification. It appears throughout cloud tooling:

Source	Generated output	Tools
OpenAPI specification	Python, Go, and Rust clients	openapi-generator, oapi-codegen, progenitor
Protocol Buffers	gRPC clients and servers	protoc, buf, tonic-build
Kubernetes API types in Go	CRD manifests, deepcopy methods, typed clients	controller-gen, client-gen
Project templates	New CLI or operator projects	kubebuilder init, cargo generate

Generated code is only as good as the specification. Review the specification for consistent naming, error schemas, and pagination before generating clients from it.

Exercises

1. Write a Rust client method that retries 429 responses, honoring Retry-After.
2. Write a Python generator that iterates over every workload across all pages.
3. Write a Go method that streams a workload's logs and stops when the context is cancelled.
4. Explain how you would version an SDK when the REST API adds a field, and when it removes one.
5. Explain how a CLI should behave when its access token expires during a long-running command.

Summary

- Design the API around resources, and make long-running operations asynchronous with 202 Accepted and an operation resource.
- An SDK handles authentication, retries with backoff and jitter, idempotency keys, typed errors, pagination, and cancellation so that callers do not have to.
- Follow each language's conventions: context arguments and options in Go, typed models in Python, Result and thiserror in Rust.
- Bound concurrency with worker pools and bounded queues, and limit request rates with token buckets.
- Generate clients from a specification to keep every language consistent.

Source: [07-cli-sdk-rest-concurrency.md](#) · [PDF](#) · [All study guides](#)